VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

JAYAM SHANMUKHA SHIVANVITHA(1BF24CS127)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICAT

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by **JAYAM SHANMUKHA SHIVANVITHA(1BF24CS127),** who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#define MAX 100
int graph[MAX][MAX];
int indegree[MAX];
int queue[MAX];
int front = 0, rear = 0;
void topologicalSort(int n){
   int i, j, count = 0;
   for(i = 0; i < n; i++){
      indegree[i] = 0;
   }
   for(i = 0; i < n; i++){
      for(j = 0; j < n; j++){
         if(graph[i][j] == 1){
            indegree[j]++;
         }
      }
   }
   for(i = 0; i < n; i++){
      if(indegree[i] == 0){
         queue[rear++] = i;
      }
   }
   printf("Topological Ordering: ");
   while(front < rear){
```

```
        int vertex = queue[front++];
        printf("%d ", vertex);
        count++;
        for(i = 0; i < n; i++){
            if(graph[vertex][i] == 1) {
                indegree[i]--;
                if(indegree[i] == 0){
                    queue[rear++] = i;
                }
            }
        }
    }
    if(count != n){
        printf("\nGraph contains a cycle. Topological ordering not possible.");
    }
}
int main(){
    int n, i, j;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++){
        for(j = 0; j < n; j++){
            scanf("%d", &graph[i][j]);
        }
    }
    topologicalSort(n);
```

```
    return 0;
}
```

Output:

```
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\week1\output> & .\'topological.exe'
Enter number of vertices: 4
Enter adjacency matrix:
0 1 0 0
0 0 1 0
0 0 0 1
0 0 0 0
Topological Ordering: 0 1 2 3
```

LEETCODE 207 Course Schedule

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you must take course b_i first if you want to take course a_i.

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

Example 1:

Input: numCourses = 2, prerequisites = [[1,0]]

Output: true

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: numCourses = 2, prerequisites = [[1,0],[0,1]]

Output: false

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

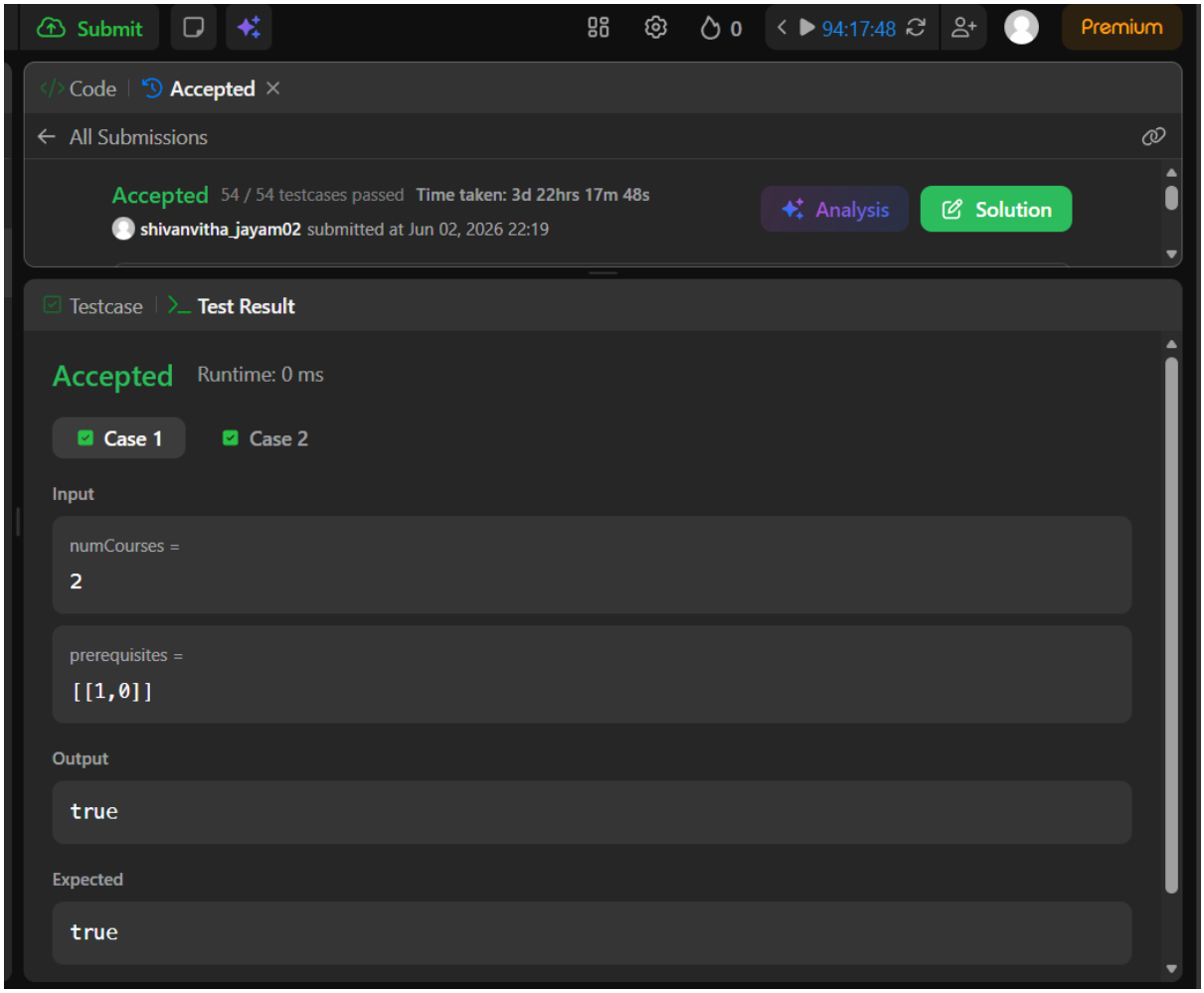
Constraints:

- 1 <= numCourses <= 2000
- 0 <= prerequisites.length <= 5000
- prerequisites[i].length == 2
- 0 <= a_i, b_i < numCourses
- All the pairs prerequisites[i] are unique

```
#include <stdbool.h>
#include <stdlib.h>
bool dfs(int course, int** graph, int* graphSize, int* visited) {
    if (visited[course] == 1) return false; // cycle detected
    if (visited[course] == 2) return true;  // already checked
 visited[course] = 1; // mark as visiting
 for (int i = 0; i < graphSize[course]; i++) {
        int next = graph[course][i];
        if (!dfs(next, graph, graphSize, visited)) return false;
    }
 visited[course] = 2; // mark as visited
    return true;
}
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* graphSize = (int*)calloc(numCourses, sizeof(int));
    int* visited = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(prerequisitesSize * sizeof(int)); // worst case
    }

    // build graph
    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        int pre = prerequisites[i][1];
        graph[pre][graphSize[pre]++] = course;
```

```
    }
    for (int i = 0; i < numCourses; i++) {
        if (!dfs(i, graph, graphSize, visited))
            for (int j = 0; j < numCourses; j++) free(graph[j]);
            free(graph);
            free(graphSize);
            free(visited);
            return false;
        }
    }
    for (int i = 0; i < numCourses; i++) free(graph[i]);
    free(graph);
    free(graphSize);
    free(visited);
    return true;
}
```

Output:


Submit
0
94:17:48
Premium
Code
Accepted
All Submissions
Accepted 54 / 54 testcases passed Time taken: 3d 22hrs 17m 48s
shivanvitha_jayam02 submitted at Jun 02, 2026 22:19
Analysis
Solution
Testcase
Test Result
Accepted Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2
prerequisites =
[[1,0]]
Output
true
Expected
true

Lab Program 2

Implement Johnson Trotter algorithm to generate permutations.

Code:

```
#include<stdio.h>
int main(){
    int n;
    printf("enter N: ");
    scanf("%d",&n);
    int p[n],dir[n];
    for(int i=0;i<n;i++){
        p[i]=i+1;
        dir[i]=-1;
    }
    while(1){
        for(int i=0;i<n;i++){
            printf("%d ",p[i]);
        }        printf("\n");
        int mobile=0,pos=-1;
        for(int i=0;i<n;i++){
            if((dir[i]==-1 && i>0 && p[i]>p[i-1]) || (dir[i]==1 && i<n-1 && p[i]>p[i+1])){
                if(p[i]>mobile){
                    mobile=p[i];
                    pos=i;
                }
            }
        }
```

```
        if(mobile==0){
        break;
       }
       int next=pos+dir[pos];
       int temp=p[pos];
       p[pos]=p[next];
       p[next]=temp;
       int tempDir = dir[pos];
       dir[pos] = dir[next];
       dir[next] = tempDir;

       pos=next;
       for(int i=0;i<n;i++){
           if(p[i]>mobile){
           dir[i]=-dir[i];
           }
        }
    }
    return 0;
}
```

Output:

```
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week2\output> & .\'johnsontrotter.exe'
enter N: 4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week2\output>
```

Lab program 3

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

Code:

```
#include<stdio.h>
#include<time.h>
#include<stdlib.h>
void merge(int beg,int mid,int end,int a[]){
    int n1=mid-beg+1;
    int n2=end-mid;
    int a1[n1],a2[n2];
    for(int i=0;i<n1;i++){
        a1[i]=a[beg+i];
    }
    for(int j=0;j<n2;j++){
        a2[j]=a[mid+1+j];
    }
    int i=0;
    int j=0;
    int k=beg;
    while(i<n1 && j<n2){
        if(a1[i]<=a2[j]){
            a[k]=a1[i];
            i++;
        }
        else{
            a[k]=a2[j];
```

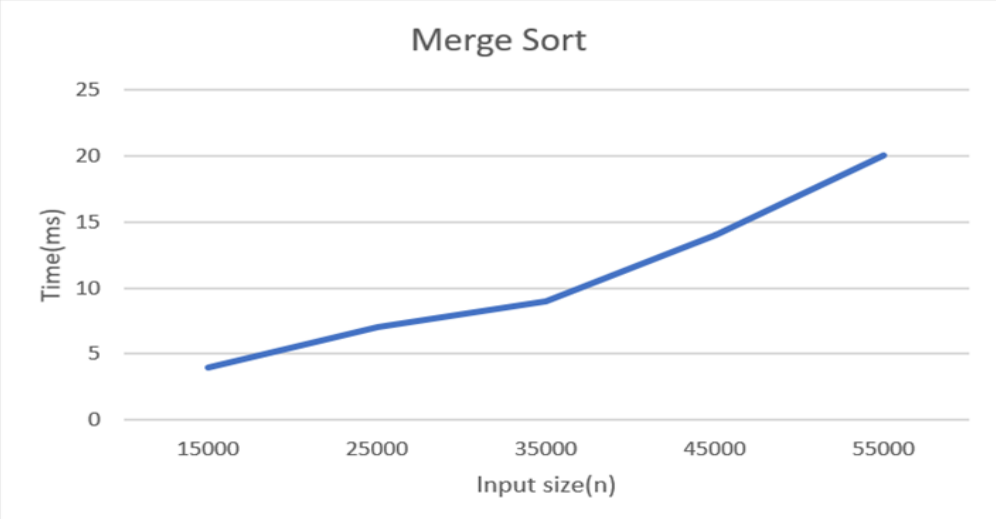
```
            j++;
        }
        k++;
    }
    while(i<n1){
        a[k]=a1[i];
        i++;
        k++;
    }
    while(j<n2){
        a[k]=a2[j];
        j++;
        k++;
    }

}
void merge_sort(int beg,int end,int a[]){
    if(beg<end){
        int mid=(beg+end)/2;
        merge_sort(beg,mid,a);
        merge_sort(mid+1,end,a);
        merge(beg,mid,end,a);
    }
}
int main(){
    int a[25000];
    srand(time(0));
```

```
    for (int i = 0; i < 25000; i++) {
        a[i] = rand()%25000;
    }
    clock_t start=clock();
    merge_sort(0,24999,a);
    clock_t end=clock();
    printf("time taken  to execute in ms is %f\n ",(double)(end-
start)*1000.0/CLOCKS_PER_SEC);
    for(int i=0;i<10;i++){
        printf("%d\t",a[i]);
    }

}
```

Output:

```
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week3\output> & .\'merge_sort.exe'
time taken  to execute in ms is 6.000000
 0      0       1       1       3       3       5       5       6       8
```

Graph:



Leetcode- 88 Merge Sorted Array

You are given two integer arrays nums1 and nums2, sorted in non-decreasing order, and two integers m and n, representing the number of elements in nums1 and nums2 respectively.

Merge nums1 and nums2 into a single array sorted in non-decreasing order.

The final sorted array should not be returned by the function, but instead be stored inside the array nums1. To accommodate this, nums1 has a length of m + n, where the first m elements denote the elements that should be merged, and the last n elements are set to 0 and should be ignored. nums2 has a length of n.

Example 1:

Input: nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3

Output: [1,2,2,3,5,6]

Explanation: The arrays we are merging are [1,2,3] and [2,5,6].

The result of the merge is [1,2,2,3,5,6] with the underlined elements coming from nums1.

Example 2:

Input: nums1 = [1], m = 1, nums2 = [], n = 0

Output: [1]

Explanation: The arrays we are merging are [1] and [].

The result of the merge is [1].

Example 3:

Input: nums1 = [0], m = 0, nums2 = [1], n = 1

Output: [1]

Explanation: The arrays we are merging are [] and [1].

The result of the merge is [1].

Note that because m = 0, there are no elements in nums1. The 0 is only there to ensure the merge result can fit in nums1.

Constraints:

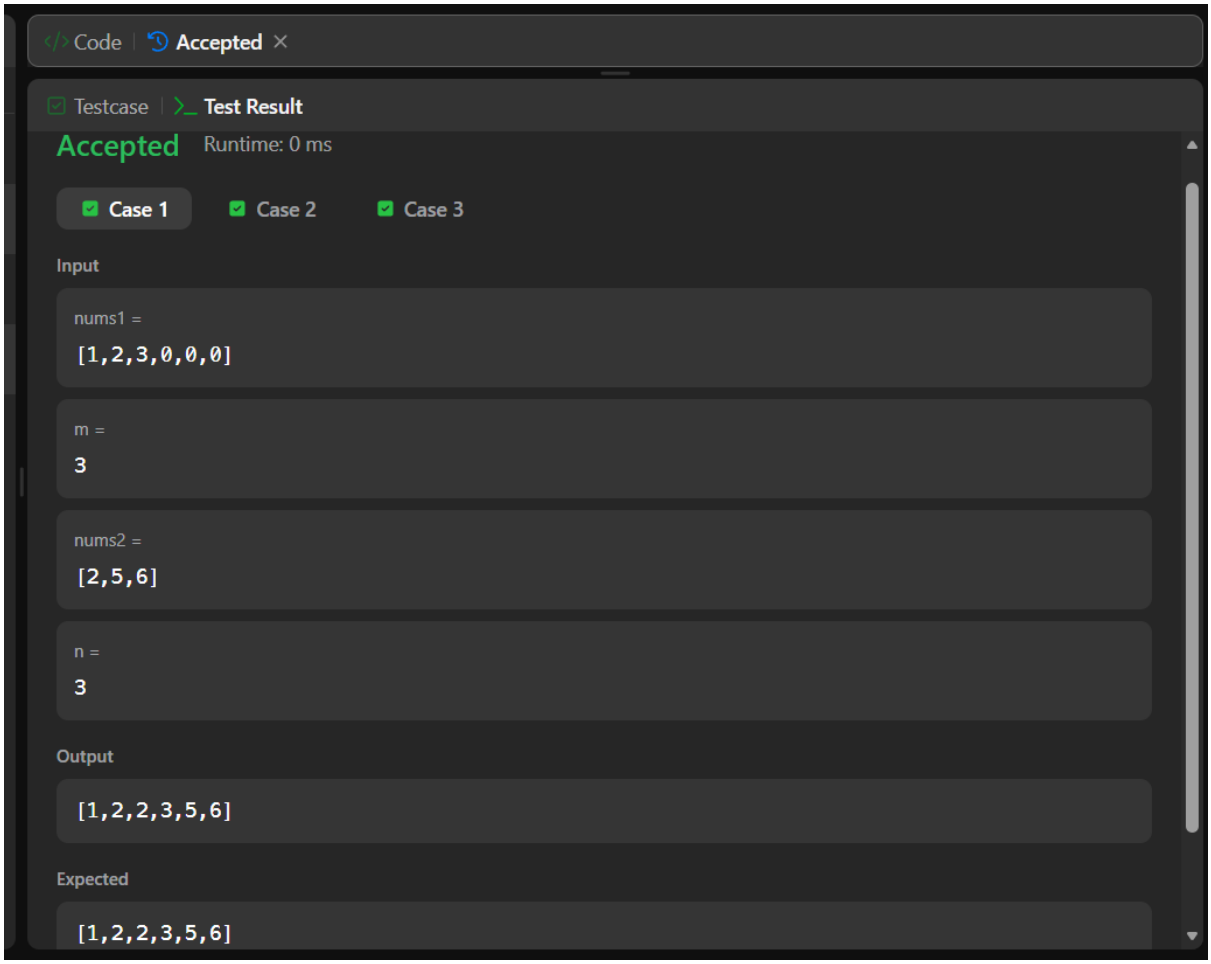
- nums1.length == m + n
- nums2.length == n
- 0 <= m, n <= 200
- 1 <= m + n <= 200
- -10^9 <= nums1[i], nums2[j] <= 10

Code:

```
void merge(int* nums1, int nums1Size, int m, int* nums2, int nums2Size, int n) {
    int i=m-1;
    int j=n-1;
    int k=m+n-1;
    while(i>=0 && j>=0){
        if(nums1[i]>nums2[j]){
            nums1[k]=nums1[i];
            i--;
        }
        else{
            nums1[k]=nums2[j];
            j--;
        }
        k--;
    }
    while(j>=0){
        nums1[k]=nums2[j];
        j--;
        k--;
    }
```

```
    while(i>=0){
        nums1[k]=nums1[i];
        i--;
        k--;
    }
}
```

Output:



Lab Program 4

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Code:

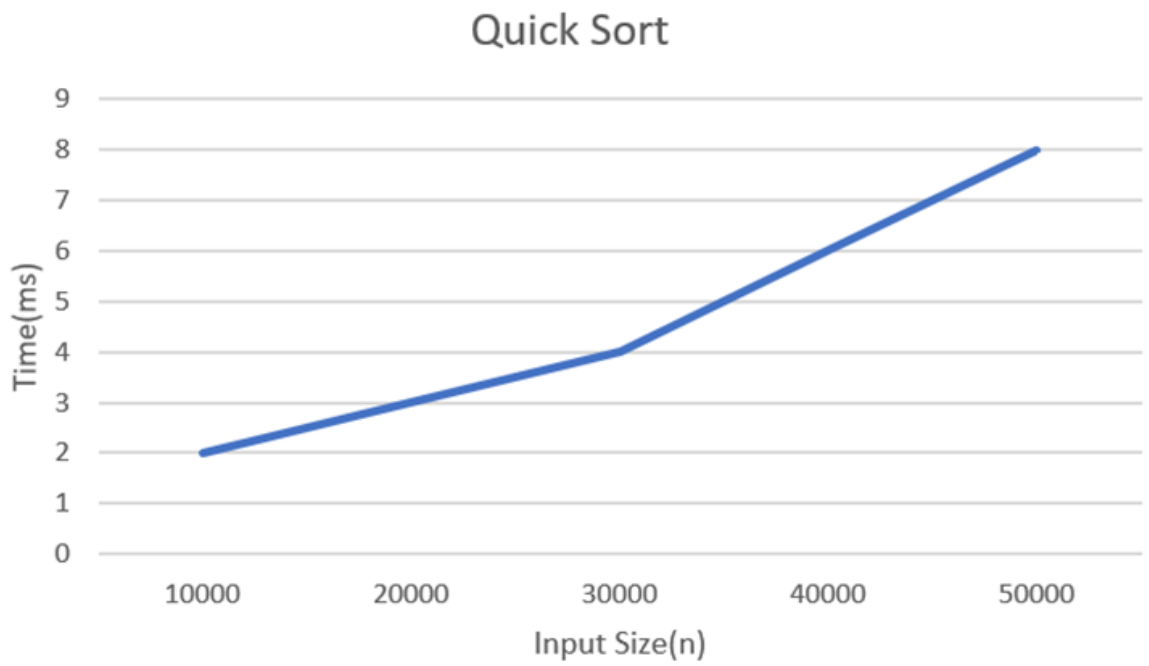
```
#include<stdio.h>
#include<time.h>
#include<stdlib.h>
void quicksort(int low,int high,int arr[]);
int partition(int low,int high,int arr[]);
void quicksort(int low,int high,int arr[]){
        if(low<high){
        int p=partition(low,high,arr);
        quicksort(low,p-1,arr);
        quicksort(p+1,high,arr);
        }
}
int partition(int low,int high,int arr[]){
        int i=low;
        int j=high+1;
        int pivot=arr[low];
        int temp;
        while(i<j){
        do{
        i++;
        }while(i<=high && arr[i]<=pivot);
        do{
        j--;
        }while(arr[j]>pivot);
        if(i<j){
        temp=arr[i];
        arr[i]=arr[j];
        arr[j]=temp;
        }
        }
        temp=arr[j];
        arr[j]=pivot;
        arr[low]=temp;
        return j;
}
void main()
{
        clock_t start,end;
        double cpu_time_used;
        int n;
        printf("Enter number of elements:");
        scanf("%d",&n);
```

```
        int arr[n];
        printf("Enter elements:");
        for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
        }
        start=clock();
        quicksort(0,n-1,arr);
        end=clock();

    cpu_time_used=(((double)(end-start))/CLOCKS_PER_SEC)*1000.0;
        printf("Sorted array:");
        for(int i=0;i<n;i++){
        printf("%d ",arr[i]);
        }
        printf("\nExecution time:%.5f ms",cpu_time_used);
}
```

Output:

```
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week4\output> & .\'quicksort.exe'
Enter number of elements:5
Enter elements:1 7 6 9 3
Sorted array:1 3 6 7 9
Execution time:0.00000 ms
```

Graph:



Leetcode 1636. Sort Array by Increasing Frequency

Given an array of integers nums, sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order.

Return the sorted array.

Example 1:

Input: nums = [1,1,2,2,2,3]

Output: [3,1,1,2,2,2]

Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.

Example 2:

Input: nums = [2,3,1,3,2]

Output: [1,3,3,2,2]

Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.

Example 3:

Input: nums = [-1,1,-6,4,5,-6,1,4,1]

Output: [5,-1,4,4,-6,-6,1,1,1]

Constraints:

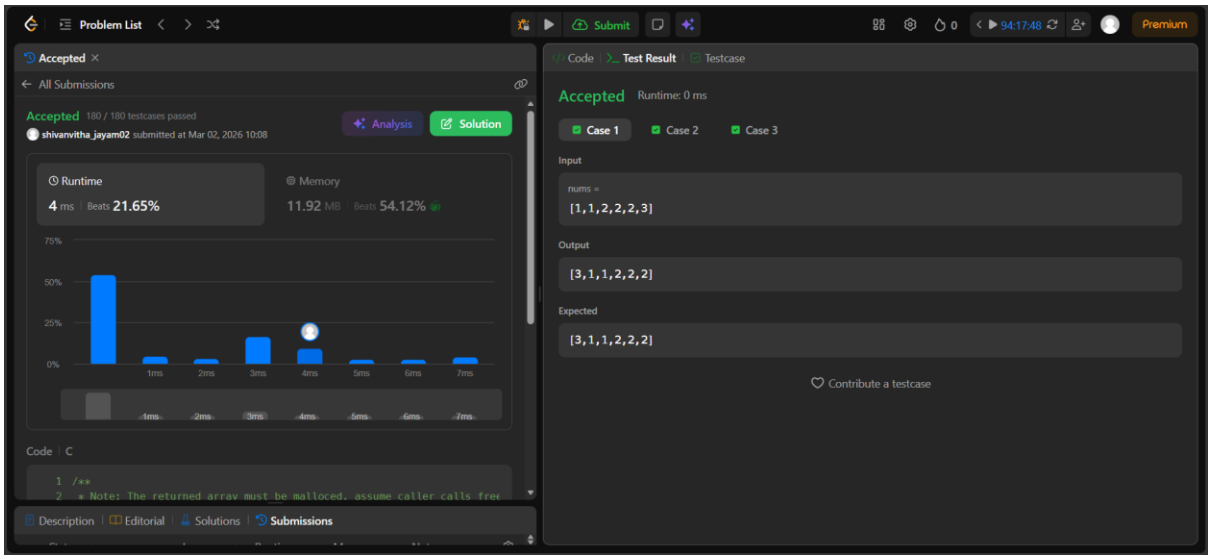
- 1 <= nums.length <= 100
- -100 <= nums[i] <= 100

Code:

```
struct node{
    int value;
    int frequency;
};
int* frequencySort(int* nums, int numsSize, int* returnSize) {
    struct node nodes[numsSize];
    for(int i=0;i<numsSize;i++){
        int count=1;
        for(int j=0;j<numsSize;j++){
            if((nums[i]==nums[j])&&(i!=j)){
                count++;
            }
        }
        nodes[i].value=nums[i];
        nodes[i].frequency=count;
    }
    for (int i = 0; i < numsSize - 1; i++) {
        for (int j = 0; j < numsSize - i - 1; j++) {
            if (nodes[j].frequency > nodes[j + 1].frequency ||
               (nodes[j].frequency == nodes[j + 1].frequency &&
                nodes[j].value < nodes[j + 1].value)) {
                // Swap
                struct node temp = nodes[j];
                nodes[j] = nodes[j + 1];
```

```
            nodes[j + 1] = temp;
        }
    }
}
int *result=(int*)malloc(numsSize*sizeof(int));
for(int i=0;i<numsSize;i++){
    result[i]=nodes[i].value;
}
*returnSize=numsSize;
return result;

}
```

Output:



Lab Program 5

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Code:

```
#include <stdio.h>

#include <time.h>

void heapify(int arr[], int n, int i) {

    int largest = i;

    int left = 2*i + 1;

    int right = 2*i + 2;

    if (left < n && arr[left] > arr[largest])

        largest = left;

    if (right < n && arr[right] > arr[largest])

        largest = right;

    // If largest is not root

    if (largest != i) {

        int temp = arr[i];

        arr[i] = arr[largest];

        arr[largest] = temp;
```

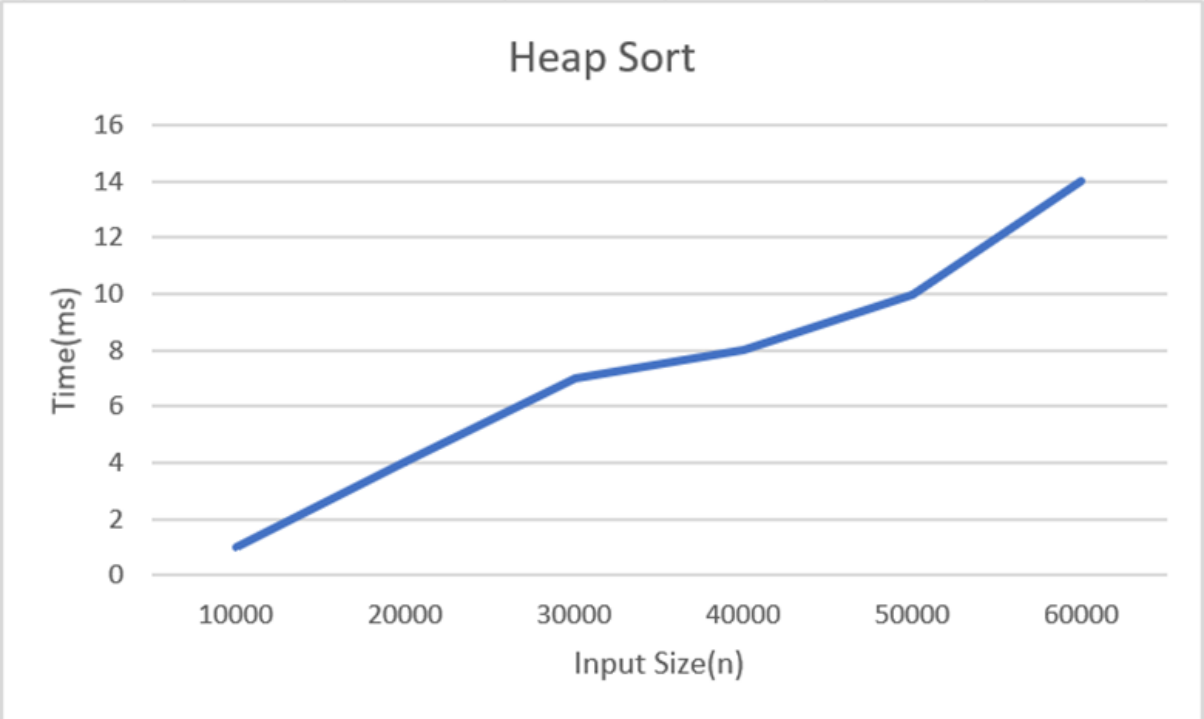
```
        heapify(arr, n, largest);
    }

void heapSort(int arr[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n-1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];"
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
int main() {
    int n;
```

```
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    clock_t start, end;
    start = clock();
    heapSort(arr, n);
    end = clock(); // end time
    printf("Sorted array:\n");
    printArray(arr, n);
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Time taken: %f seconds\n", time_taken);
    return 0;
}
```

Output:

```
Enter number of elements: 6
Enter elements:
3 7 6 1 9 2
Sorted array:
1 2 3 6 7 9
Time taken: 0.000000 seconds
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week5\output>
```

Graph:



Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include<stdio.h>
int max(int a,int b){
   if(a>b)
      return a;
   else
      return b;
}
int knapsack(int W,int wt[],int val[],int n){
   if(n==0||W==0)
      return 0;
   if(wt[n-1]>W)
      return knapsack(W,wt,val,n-1);
   else
      return max(val[n-1]+knapsack(W-wt[n-1],wt,val,n-1),knapsack(W,wt,val,n-1));
}
int main(){
   int val[]={3,4,5,6};
   int wt[]={2,3,4,5};
   int W=8;
   int n=sizeof(val)/sizeof(val[0]);
   printf("%d",knapsack(W,wt,val,n));
   return 0;
}
```

Output:

```
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week6\0\output> & .\'1knapsack.exe'
10
```

Leetcode 821 Shortest distance to a character

Given a string s and a character c that occurs in s, return an array of integers answer where answer.length == s.length and answer[i] is the distance from index i to the closest occurrence of character c in s.

The distance between two indices i and j is abs(i - j), where abs is the absolute value function.

Example 1:

Input: s = "loveleetcode", c = "e"

Output: [3,2,1,0,1,0,0,1,2,2,1,0]

Explanation: The character 'e' appears at indices 3, 5, 6, and 11 (0-indexed).

The closest occurrence of 'e' for index 0 is at index 3, so the distance is abs(0 - 3) = 3.

The closest occurrence of 'e' for index 1 is at index 3, so the distance is abs(1 - 3) = 2.

For index 4, there is a tie between the 'e' at index 3 and the 'e' at index 5, but the distance is still the same: abs(4 - 3) == abs(4 - 5) = 1.

The closest occurrence of 'e' for index 8 is at index 6, so the distance is abs(8 - 6) = 2.

Example 2:

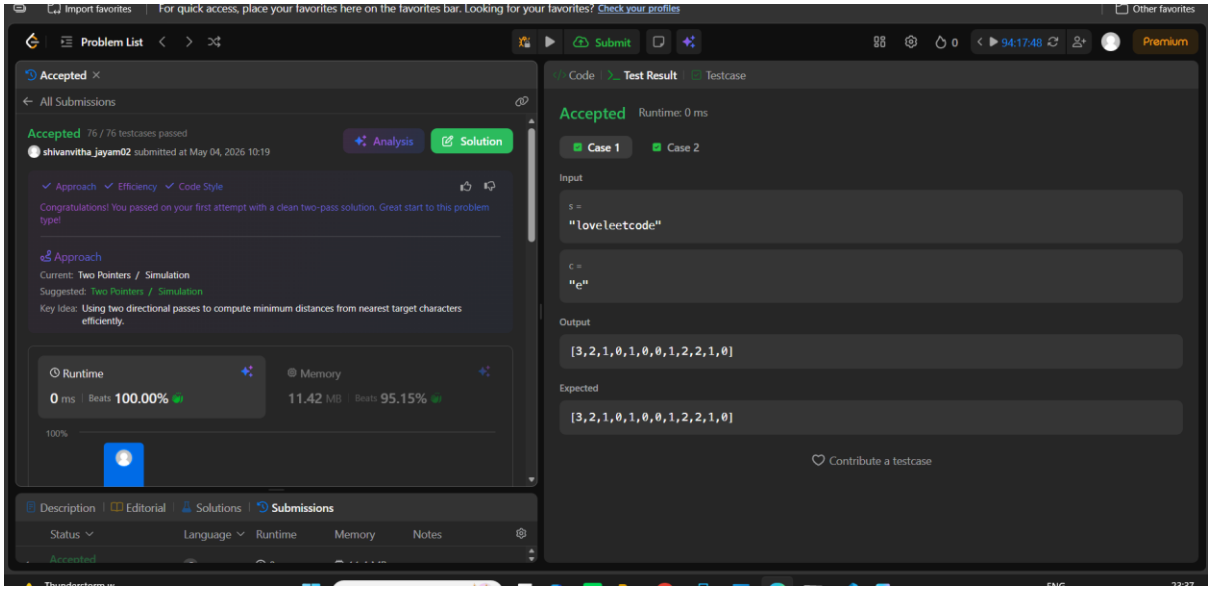
Input: s = "aaab", c = "b"

Output: [3,2,1,0]

Constraints:

- 1 <= s.length <= 104
- s[i] and c are lowercase English letters.
- It is guaranteed that c occurs at least once in s.

```
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* shortestToChar(char* s, char c, int* returnSize) {
     int n = strlen(s);
    *returnSize = n;
    int *ans = (int*)malloc(n * sizeof(int));
int prev = -n;
    for (int i = 0; i < n; i++) {
        if (s[i] == c) prev = i;
        ans[i] = i - prev;
    }
    prev = 2 * n; // something very large
    for (int i = n - 1; i >= 0; i--) {
        if (s[i] == c) prev = i;
        if (prev - i < ans[i]) ans[i] = prev - i;
    }
    return ans;



}
```


Problem List
Submit
Premium
Accepted
All Submissions
Accepted 76 / 76 testcases passed
shivanvitha_jayam02 submitted at May 04, 2026 10:19
Analysis
Solution
Approach
Efficiency
Code Style
Congratulations! You passed on your first attempt with a clean two-pass solution. Great start to this problem type!
Approach
Current: Two Pointers / Simulation
Suggested: Two Pointers / Simulation
Key Idea: Using two directional passes to compute minimum distances from nearest target characters efficiently.
Runtime
0 ms | Beats 100.00%
Memory
11.42 MB | Beats 95.15%
Description
Editorial
Solutions
Submissions
Status
Language
Runtime
Memory
Notes
Code
Test Result
Testcase
Accepted Runtime: 0 ms
Case 1
Case 2
Input
s =
"loveleetcode"
c =
"e"
Output
[3,2,1,0,1,0,0,1,2,2,1,0]
Expected
[3,2,1,0,1,0,0,1,2,2,1,0]
Contribute a testcase

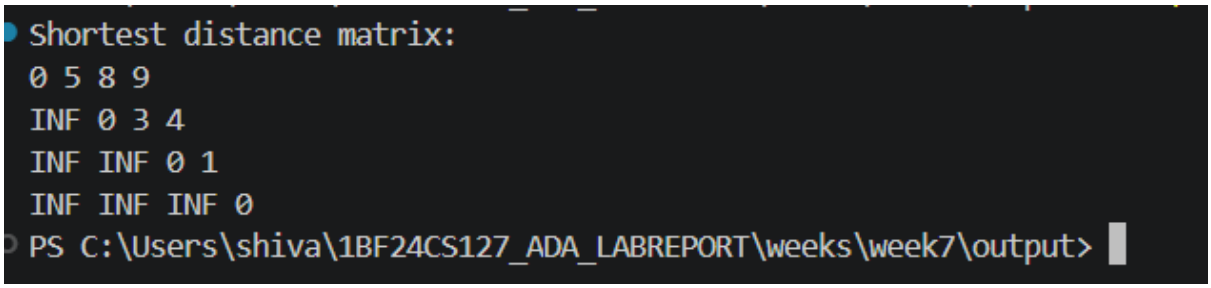
Lab program 7

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>

#define V 4

#define INF 99999

void floydWarshall(int graph[V][V]) {
   int dist[V][V];
   for (int i = 0; i < V; i++)
      for (int j = 0; j < V; j++)
         dist[i][j] = graph[i][j];
   for (int k = 0; k < V; k++) {
      for (int i = 0; i < V; i++) {
         for (int j = 0; j < V; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
               dist[i][j] = dist[i][k] + dist[k][j];
            }
         }
      }
   }
```

```
    printf("Shortest distance matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
        printf("\n");
    }
int main() {
    int graph[V][V] = {
        {0,   5,  INF, 10},
        {INF, 0,   3, INF},
        {INF, INF, 0,   1},
        {INF, INF, INF, 0}
    };

    floydWarshall(graph);
    return 0;
}
```

output:


Shortest distance matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week7\output>

Leetcode 70. Climbing Stairs

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: n = 2

Output: 2

Explanation: There are two ways to climb to the top.

1. 1 step + 1 step

2. 2 steps

Example 2:

Input: n = 3

Output: 3

Explanation: There are three ways to climb to the top.

1. 1 step + 1 step + 1 step

2. 1 step + 2 steps

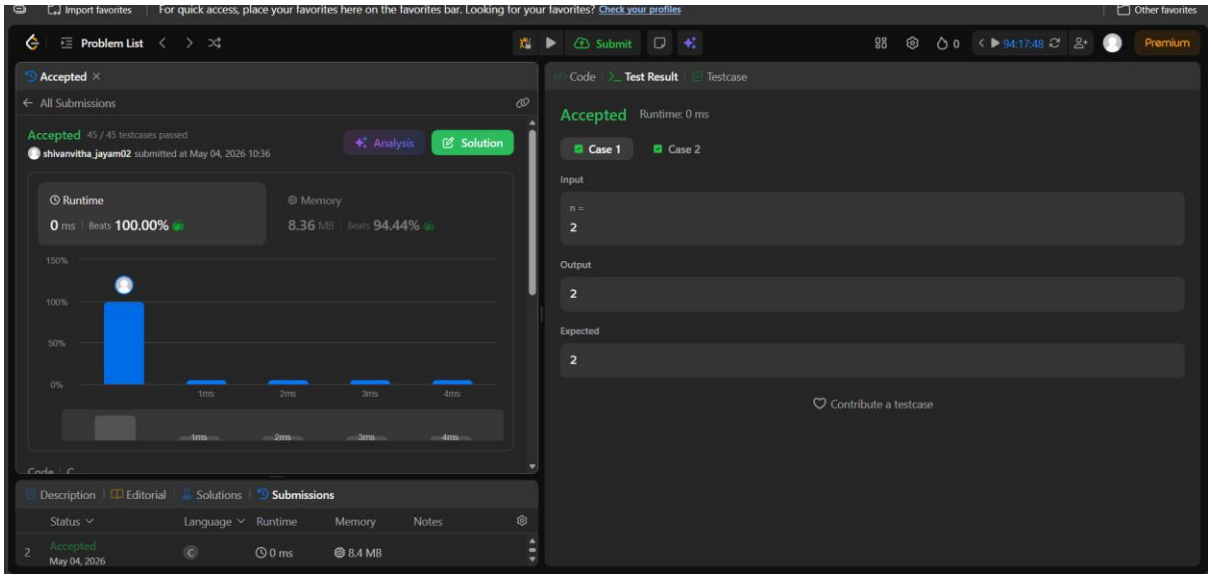
3. 2 steps + 1 step

Constraints:

- 1 <= n <= 45

```
int climbStairs(int n)

{

   if (n == 0 || n == 1) {

        return 1;

    }

    int prev = 1, curr = 1;

    for (int i = 2; i <= n; i++) {

        int temp = curr;

        curr = prev + curr;

        prev = temp;

    }

    return curr;

}
```

Output:


Problem List
Submit
Premium
Accepted
All Submissions
Accepted 45 / 45 testcases passed
shivanvitha_jayam02 submitted at May 04, 2026 10:36
Analysis
Solution
Runtime
0 ms | Beats 100.00%
Memory
8.36 MB | Beats 94.44%
150%
100%
50%
0%
1ms
2ms
3ms
4ms
Description
Editorial
Solutions
Submissions
Status
Language
Runtime
Memory
Notes
Accepted
May 04, 2026
0 ms
8.4 MB
Code
Test Result
Testcase
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
n =
2
Output
2
Expected
2
Contribute a testcase

Lab Program 8

8a)Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (5 Marks)

```
#include <stdio.h>

#include <limits.h>

#define V 5

int minKey(int key[], int mstSet[]) {

   int min = INT_MAX, min_index;

   for (int v = 0; v < V; v++) {

      if (mstSet[v] == 0 && key[v] < min) {

         min = key[v];

         min_index = v;

      }

   }

   return min_index;

}

void printMST(int parent[], int graph[V][V]) {

   printf("Edge   Weight\n");

   for (int i = 1; i < V; i++)

      printf("%d - %d    %d\n", parent[i], i, graph[i][parent[i]]);

}
```

```
void primMST(int graph[V][V]) {
    int parent[V];   // Array to store constructed MST
    int key[V];
    int mstSet[V];
    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet);
        mstSet[u] = 1;
        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
```

```
    printMST(parent, graph);
}

int main() {
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };

    primMST(graph);
return 0;
}
```

Output:

```
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week8\output> & .\'prims.exe'
Edge   Weight
0 - 1    2
1 - 2    3
0 - 3    6
1 - 4    5
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week8\output>
```

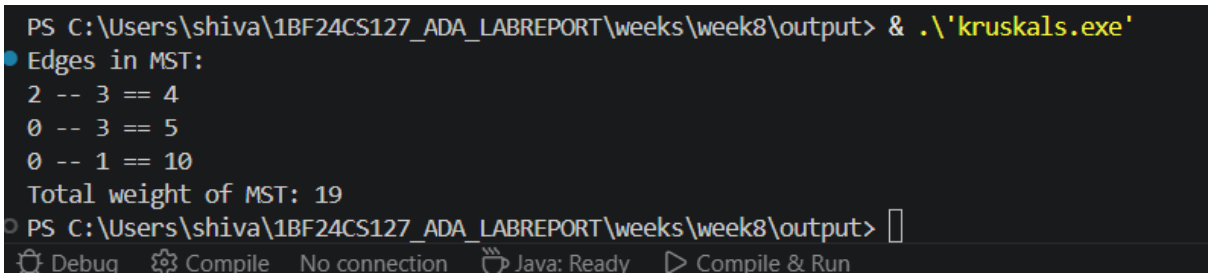
8b)Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm. (5 Marks)

```
#include <stdio.h>
#include <stdlib.h>
struct Edge {
    int u, v, w;
};
int find(int parent[], int i) {
    if (parent[i] == i) return i;
    return find(parent, parent[i]);
}
int cmp(const void* a, const void* b) {
    return ((struct Edge*)a)->w - ((struct Edge*)b)->w;
}
int main() {
    int V = 4;
    int E = 5;
    struct Edge edges[] = {
        {0, 1, 10},
        {0, 2, 6},
        {0, 3, 5},
```

```
    {1, 3, 15},
    {2, 3, 4}
};
qsort(edges, E, sizeof(edges[0]), cmp);
int parent[V];
for (int i = 0; i < V; i++) parent[i] = i;
printf("Edges in MST:\n");
int totalWeight = 0, count = 0;
for (int i = 0; i < E && count < V - 1; i++) {
    int u = edges[i].u;
    int v = edges[i].v;
    int w = edges[i].w;
    int pu = find(parent, u);
    int pv = find(parent, v);
    if (pu != pv) {
        printf("%d -- %d == %d\n", u, v, w);
        totalWeight += w;
        parent[pu] = pv; // simple union
        count++;
    }
}
```

```
    printf("Total weight of MST: %d\n", totalWeight);

    return 0;

}
```

Output:



Lab Program 9

Implement Fractional Knapsack using Greedy technique.

```
#include<stdio.h>

#include<stdlib.h>

#define max 100

float knapsack(float capacity,float weight[max],float profit[max],float ratio[max],int n){

   int i=0;

   float total_profit=0;

   while(i<n && capacity>0){

      if(weight[i]<=capacity){

         capacity-=weight[i];

         total_profit+=profit[i];

         i++;

      }

      else{

         total_profit+=ratio[i]*capacity;

         break;

      }

   }

   return total_profit;
```

```
}
float weight[max],profit[max],ratio[max];
int main(){
    int n;
    float capacity;
    printf("enter number of objects:");
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        printf("enter profit of obj %d:",i);
        scanf("%f",&profit[i]);
        printf("enter weight of obj %d:",i);
        scanf("%f",&weight[i]);
        ratio[i]=profit[i]/weight[i];
    }
    int swapped;
    float temp;
    do{
        swapped=0;
        for(int i=0;i<n-1;i++){
            if(ratio[i]<ratio[i+1]){
                temp=ratio[i];
```

```
            ratio[i]=ratio[i+1];
            ratio[i+1]=temp;
            temp=profit[i];
            profit[i]=profit[i+1];
            profit[i+1]=temp;
            temp=weight[i];
            weight[i]=weight[i+1];
            weight[i+1]=temp;
            swapped=1;
        }
    }
}while(swapped);
printf("enter capacity:");
scanf("%f",&capacity);
float total_profit=knapsack(capacity,weight,profit,ratio,n);
printf("Total profit = %.2f\n", total_profit);
return 0;
}o
```

Output:

```
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week9\output> & .\'knapsack.exe'
enter number of objects:3
enter profit of obj 0:60
enter weight of obj 0:10
enter profit of obj 1:100
enter weight of obj 1:20
enter profit of obj 2:120
enter weight of obj 2:30
enter capacity:50
Total profit = 240.00
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week9\output>
```

Leetcode 455. Assign Cookies

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input: g = [1,2,3], s = [1,1]

Output: 1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Example 2:

Input: g = [1,2], s = [1,2,3]

Output: 2

Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1, 2.

You have 3 cookies and their sizes are big enough to gratify all of the children,

You need to output 2.

Constraints:

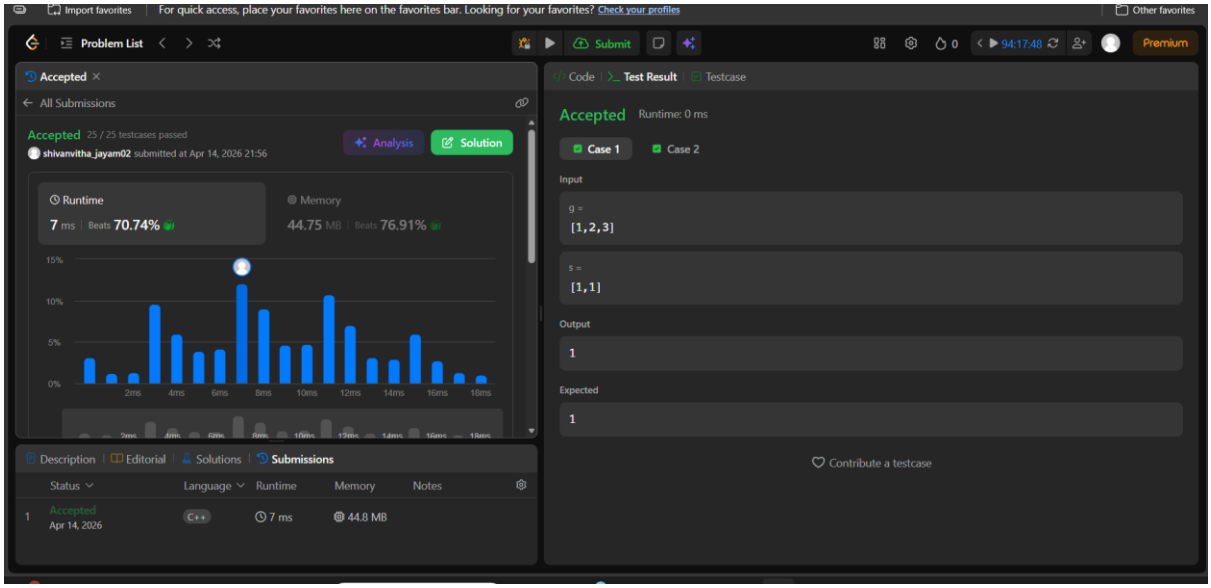
- 1 <= g.length <= 3 * 104

- 0 <= s.length <= 3 * 104
- 1 <= g[i], s[j] <= 231 - 1

Code:

```
int compare(const void *a, const void *b) {
return (*(int *)a - *(int *)b);
}

int findContentChildren(int* g, int gSize, int* s, int sSize) {
qsort(g, gSize, sizeof(int), compare);
qsort(s, sSize, sizeof(int), compare);

int i = 0, j = 0, count = 0;

while (i < gSize && j < sSize) {
if (g[i] <= s[j]) {
count++;
i++;
}
j++;
}

return count;
}
```

Output:


Problem List
Submit
Premium
Accepted
All Submissions
Accepted 25 / 25 testcases passed
shivanvitha_jayam02 submitted at Apr 14, 2026 21:56
Analysis
Solution
Runtime
7 ms | Beats 70.74%
Memory
44.75 MB | Beats 76.91%
Description | Editorial | Solutions | Submissions
Status
Language
Runtime
Memory
Notes
1
Accepted
Apr 14, 2026
C++
7 ms
44.8 MB
Code | Test Result | Testcase
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
g =
[1,2,3]
s =
[1,1]
Output
1
Expected
1
Contribute a testcase

Lab program 10

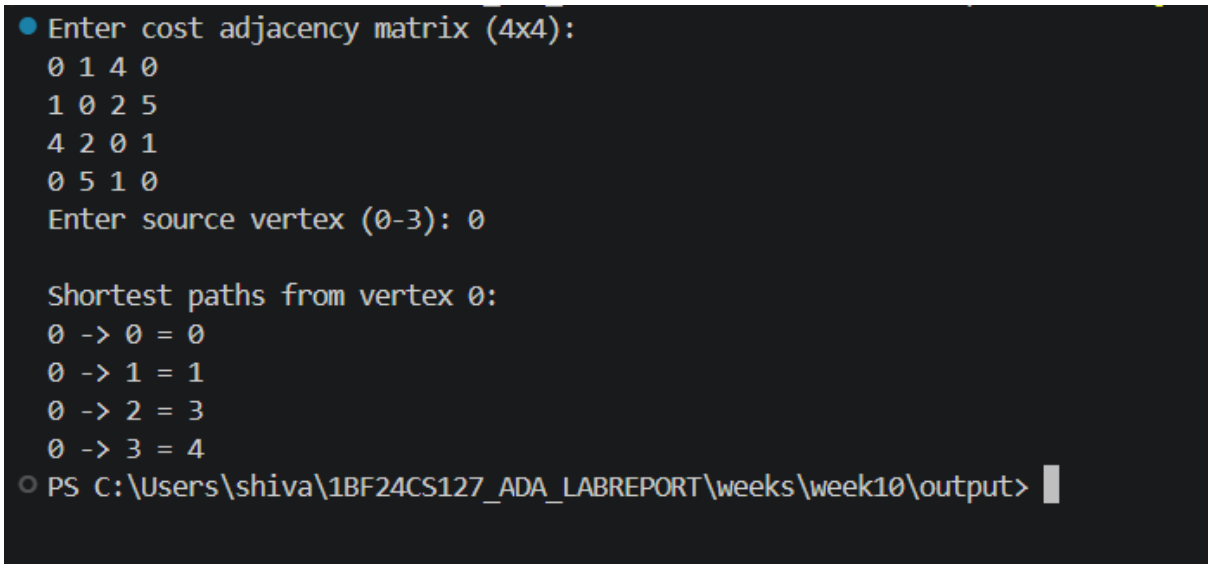
From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>

#include <stdlib.h>

#include <limits.h>

#define INF 999

#define n 5

int min(int d[], int visited[]) {

   int min = INT_MAX;

   int min_index = -1;

   for (int i = 0; i < n; i++) {

      if (visited[i] == 0 && d[i] <= min) {

         min = d[i];

         min_index = i;

      }

   }

   return min_index;

}



void dijkstra(int cost[n][n], int source) {
```

```
int dist[n];
int visited[n];
for (int i = 0; i < n; i++) {
    dist[i] = INT_MAX;
    visited[i] = 0;
}
dist[source] = 0;
for (int j = 0; j < n - 1; j++) {
    int u = min(dist, visited);
    visited[u] = 1;

    for (int v = 0; v < n; v++) {
        if (visited[v] != 1 && cost[u][v] != INF && dist[u] != INT_MAX &&
            dist[u] + cost[u][v] < dist[v]) {
            dist[v] = dist[u] + cost[u][v];
        }
    }
}
printf("\nShortest paths from vertex %d:\n", source);
for (int i = 0; i < n; i++) {
    printf("%d -> %d = %d\n", source, i, dist[i]);
```

```
    }
}
int main() {
    int i, j, source;
    int cost[n][n];
    printf("Enter cost adjacency matrix (%dx%d):\n", n, n);
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0 && i != j)
                cost[i][j] = INF;  // Replace 0 with INF for non-edges
        }
    }
    printf("Enter source vertex (0-%d): ", n - 1);
    scanf("%d", &source);
    dijkstra(cost, source);
    return 0;
}
```

Output:


Enter cost adjacency matrix (4x4):
0 1 4 0
1 0 2 5
4 2 0 1
0 5 1 0
Enter source vertex (0-3): 0
Shortest paths from vertex 0:
0 -> 0 = 0
0 -> 1 = 1
0 -> 2 = 3
0 -> 3 = 4
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week10\output>

Lab program 11

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>

int board[20];

int n;

int isSafe(int row, int col) {
    for (int i = 1; i < row; i++) {
        if (board[i] == col ||
            (i - board[i] == row - col) ||
            (i + board[i] == row + col))
            return 0;
    }
    return 1;
}

void solve(int row) {
    if (row > n) {
        printf("\nSolution:\n");
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (board[i] == j)
```

```
                printf("Q ");
            else
                printf(". ");
        }
        printf("\n");
    }
    return;
  }
 for (int col = 1; col <= n; col++) {
    if (isSafe(row, col)) {
        board[row] = col;
        solve(row + 1);
    }
  }
}
int main() {
  printf("Enter N: ");
  scanf("%d", &n);
  solve(1);
  return 0;
}
```

output:


PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week11\output> & .\'nqueens.exe'
Enter N: 4
Solution:
. Q . .
. . . Q
Q . . .
. . Q .
Solution:
. . Q .
Q . . .
. . . Q
. Q . .
PS C:\Users\shiva\1BF24CS127_ADA_LABREPORT\weeks\week11\output>
Debug
Compile
No connection
Java: Ready
Compile & Run